

```
import zipfile
import os

zip_path = "/content/plant_sp.zip"
extract_path = "/content/plant_sp"

with zipfile.ZipFile(zip_path, "r") as zip_ref:
    zip_ref.extractall(extract_path)

print("Dataset extracted successfully!")
```

Dataset extracted successfully!

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

dataset_path = "/content/plant_sp"

print("TensorFlow:", tf.__version__)
print("Dataset exists:", os.path.exists(dataset_path))
print("Classes:", os.listdir(dataset_path))
```

TensorFlow: 2.20.0
Dataset exists: True
Classes: ['aloevera', 'corn', 'cucumber', 'orange', 'mango', 'banana', 'melon', 'watermelon', 'guava', 'ginger', 'paddy']

```
train_data = tf.keras.utils.image_dataset_from_directory(
    dataset_path,
    validation_split=0.2,
    subset="training",
    seed=42,
    image_size=(224, 224),
    batch_size=32
)

val_data = tf.keras.utils.image_dataset_from_directory(
    dataset_path,
    validation_split=0.2,
    subset="validation",
    seed=42,
    image_size=(224, 224),
    batch_size=32
)

class_names = train_data.class_names
num_classes = len(class_names)

print("Number of classes:", num_classes)
print("Classes:", class_names)
```

Found 9800 files belonging to 14 classes.
Using 7840 files for training.
Found 9800 files belonging to 14 classes.
Using 1960 files for validation.
Number of classes: 14
Classes: ['aloevera', 'banana', 'coconut', 'corn', 'cucumber', 'ginger', 'guava', 'mango', 'melon', 'orange', 'paddy', '']

```
AUTOTUNE = tf.data.AUTOTUNE

train_data = train_data.shuffle(1000).prefetch(AUTOTUNE)
val_data = val_data.prefetch(AUTOTUNE)

print("Dataset ready!")
```

Dataset ready!

```
from tensorflow.keras import layers, models
from tensorflow.keras.applications import MobileNetV2

base_model = MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights="imagenet"
)
```

```
base_model.trainable = False

model = models.Sequential([
    layers.Input(shape=(224, 224, 3)),

    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),

    layers.Rescaling(1./255),

    base_model,

    layers.GlobalAveragePooling2D(),

    layers.Dense(128, activation="relu"),
    layers.Dropout(0.4),

    layers.Dense(num_classes, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
random_flip_2 (RandomFlip)	(None, 224, 224, 3)	0
random_rotation_2 (RandomRotation)	(None, 224, 224, 3)	0
random_zoom_2 (RandomZoom)	(None, 224, 224, 3)	0
rescaling_2 (Rescaling)	(None, 224, 224, 3)	0
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling2d_2 (GlobalAveragePooling2D)	(None, 1280)	0
dense_4 (Dense)	(None, 128)	163,968
dropout_2 (Dropout)	(None, 128)	0
dense_5 (Dense)	(None, 14)	1,806

Total params: 2,423,758 (9.25 MB)
Trainable params: 165,774 (647.55 KB)

```
history = model.fit(
    train_data,
    validation_data=val_data,
    epochs=5
)
```

Epoch 1/5
245/245 ————— 39s 77ms/step - accuracy: 0.6335 - loss: 1.2164 - val_accuracy: 0.8128 - val_loss: 0.6357
Epoch 2/5
245/245 ————— 31s 67ms/step - accuracy: 0.7790 - loss: 0.7313 - val_accuracy: 0.8332 - val_loss: 0.5442
Epoch 3/5
245/245 ————— 29s 67ms/step - accuracy: 0.8094 - loss: 0.6051 - val_accuracy: 0.8342 - val_loss: 0.5073
Epoch 4/5
245/245 ————— 30s 74ms/step - accuracy: 0.8328 - loss: 0.5259 - val_accuracy: 0.8561 - val_loss: 0.4605
Epoch 5/5
245/245 ————— 29s 67ms/step - accuracy: 0.8515 - loss: 0.4667 - val_accuracy: 0.8515 - val_loss: 0.4618

```
best_accuracy = max(history.history["val_accuracy"])

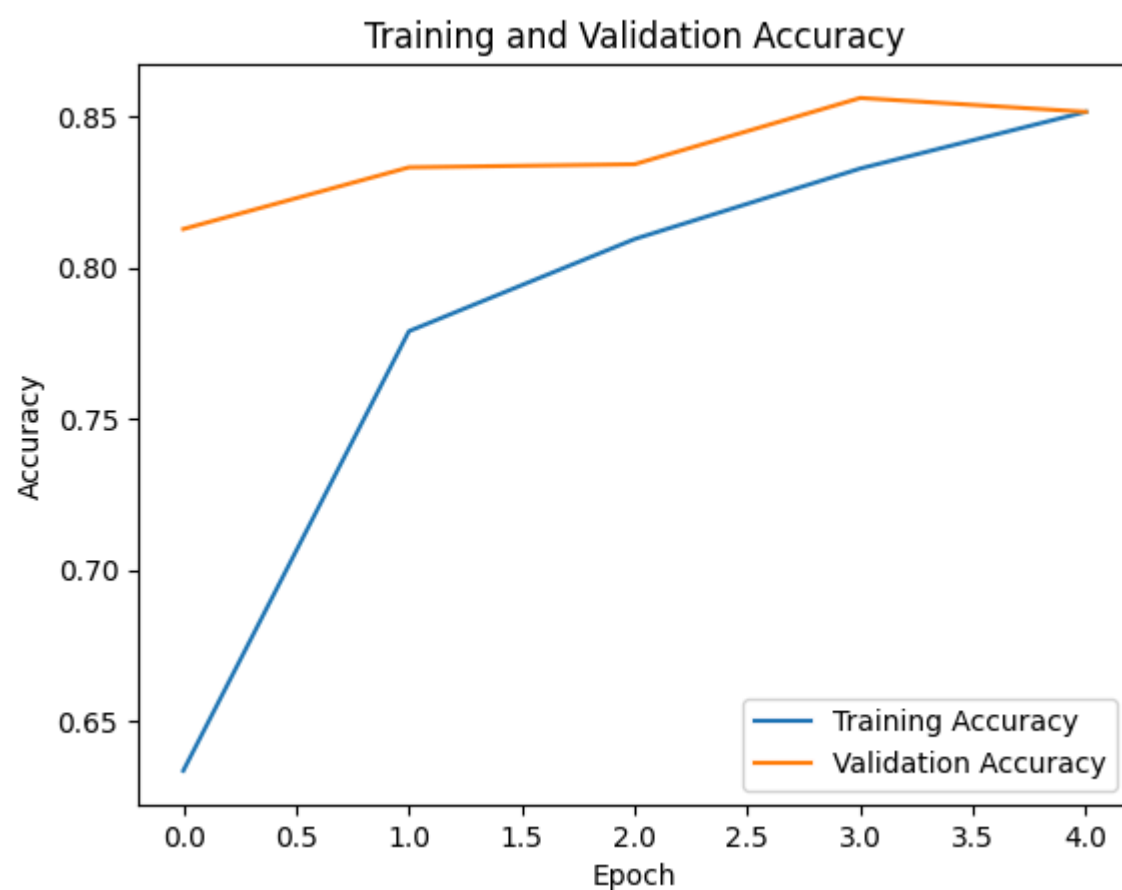
print("Best Validation Accuracy:",
      round(best_accuracy * 100, 2), "%")
```

Best Validation Accuracy: 85.61 %

```
import matplotlib.pyplot as plt

plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training and Validation Accuracy")
plt.legend()
plt.show()
```



```
loss, accuracy = model.evaluate(val_data)

print("Validation Loss:", round(loss, 4))
print("Validation Accuracy:", round(accuracy * 100, 2), "%")
```

```
62/62 ————— 4s 58ms/step - accuracy: 0.8515 - loss: 0.4618
Validation Loss: 0.4618
Validation Accuracy: 85.15 %
```

```
model.save("/content/plant_species_model.keras")

print("Model saved successfully!")
```

```
Model saved successfully!
```

```
from google.colab import files
from tensorflow.keras.utils import load_img, img_to_array

uploaded = files.upload()

img_path = next(iter(uploaded))

img = load_img(img_path, target_size=(224, 224))
img_array = img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)

predictions = model.predict(img_array)

predicted_class = np.argmax(predictions[0])
confidence = np.max(predictions[0]) * 100

print("Predicted Plant:", class_names[predicted_class])
print("Confidence:", round(confidence, 2), "%")

plt.imshow(img)
plt.axis("off")
plt.show()
```

Aloe_Vera.jpg
Aloe_Vera.jpg(image/jpeg) - 53326 bytes, last modified: 10/1/2026 - 100% done
Saving Aloe_Vera.jpg to Aloe_Vera (2).jpg
1/1 2s 2s/step
Predicted Plant: aloevera
Confidence: 98.62 %



```
top_3 = np.argsort(predictions[0])[-3:][::-1]

print("Top 3 Predictions:")

for i in top_3:
    print(
        class_names[i],
        ":", round(predictions[0][i] * 100, 2), "%"
    )
```

Top 3 Predictions:
aloevera : 98.62 %
ginger : 0.28 %
corn : 0.23 %

```
plt.figure(figsize=(6, 6))
plt.imshow(img)
plt.title(
    f"Prediction: {class_names[predicted_class]}\n"
    f"Confidence: {confidence:.2f}%"
)
plt.axis("off")
plt.show()
```

Prediction: aloe vera
Confidence: 98.62%

